

FIT3143 – LAB TASK 1

Team members:

- Natthapol Sinsirithavorn (34723684)
email: *nsin0058@student.monash.edu*
- Shivam Khadka (33453411)
email: *skha0108@student.monash.edu*

A

```
for (long long i = 2; i <= limit; i++)  
{  
    if (is_prime(i))  
    {  
        |  
    }
```

```
int is_prime(long long num)  
{  
    if (num < 2)  
        return 0;  
    if (num == 2)  
        return 1;  
    if (num % 2 == 0)   
        return 0;  
  
    for (long long i = 3; i * i <= num; i += 2)  
    {  
        if (num % i == 0)  
            return 0;  
    }  
    return 1;  
}
```

Even = not prime

If i is not a prime number, there must exist two integers m and n such that $m * n = i$. If m is larger than the square root of k , then n must be smaller than the square root of k . So we only need to check until square root of i .

B

```
long long chunk_size = (limit - 1) / num_threads;  
for (int t = 0; t < num_threads; t++)  
{
```

Distribute chunk of all numbers into equal chunks for each thread

```
    // the rest gets the same amount  
    thread_args[t].end_val = thread_args[t].start_val + chunk_size - 1;  
}  
  
// set argument to parse into function called by each thread  
thread_args[t].primes = &primes[thread_offset];  
thread_args[t].local_count = &local_counts[t];  
thread_args[t].limit = limit;  
  
// Reserve enough space for this thread's entire input range.  
thread_offset += thread_args[t].end_val - thread_args[t].start_val + 1;  
  
pthread_create(&threads[t], NULL, check_prime, &thread_args[t]);  
}
```

Store global result

Store primes counts in each thread

Create thread, pass smaller chunk of numbers into each thread

```
void *check_prime(void *arg)
```

```
// local primes array for storing primes in this thread (will be added
long long range_size = data->end_val - data->start_val + 1;
long long *local_primes = malloc(range_size * sizeof(long long));
long long local_c = 0;
```

store primes in local array

```
// loop to check all numbers in the assigned range in this thread
for (long long i = data->start_val; i <= data->end_val; i++)
{
    if (is_prime(i))
    {
        local_primes[local_c++] = i;
    }
}
```

add local array

```
*(data->local_count) = local_c;
```

```
// add local_primes back to primes
memcpy(data->primes, local_primes, local_c * sizeof(long long));
```

copy back from local to “primes”

Remove all the empty spaces from main array
since we combine all local arrays to global array with
additional offset for safety

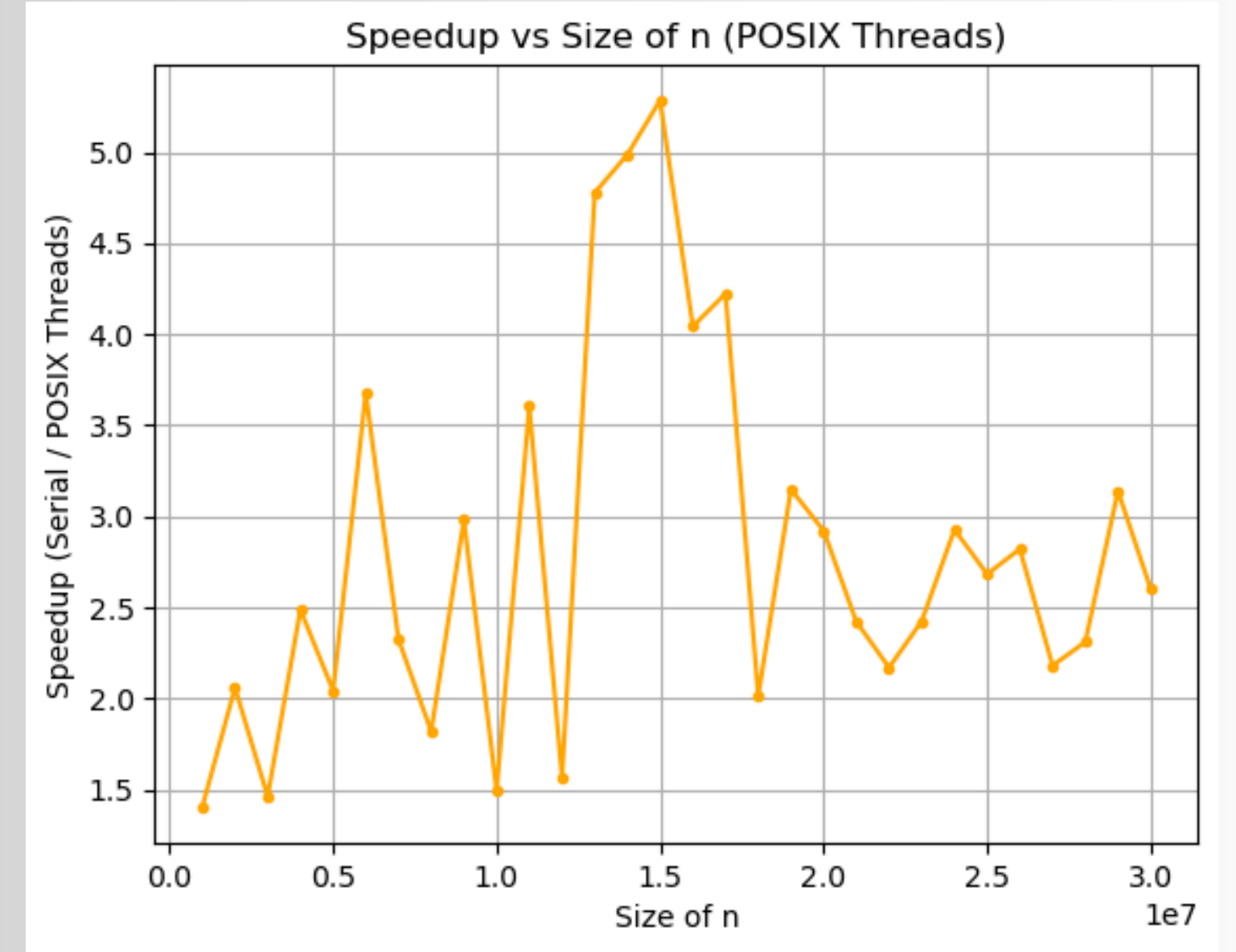
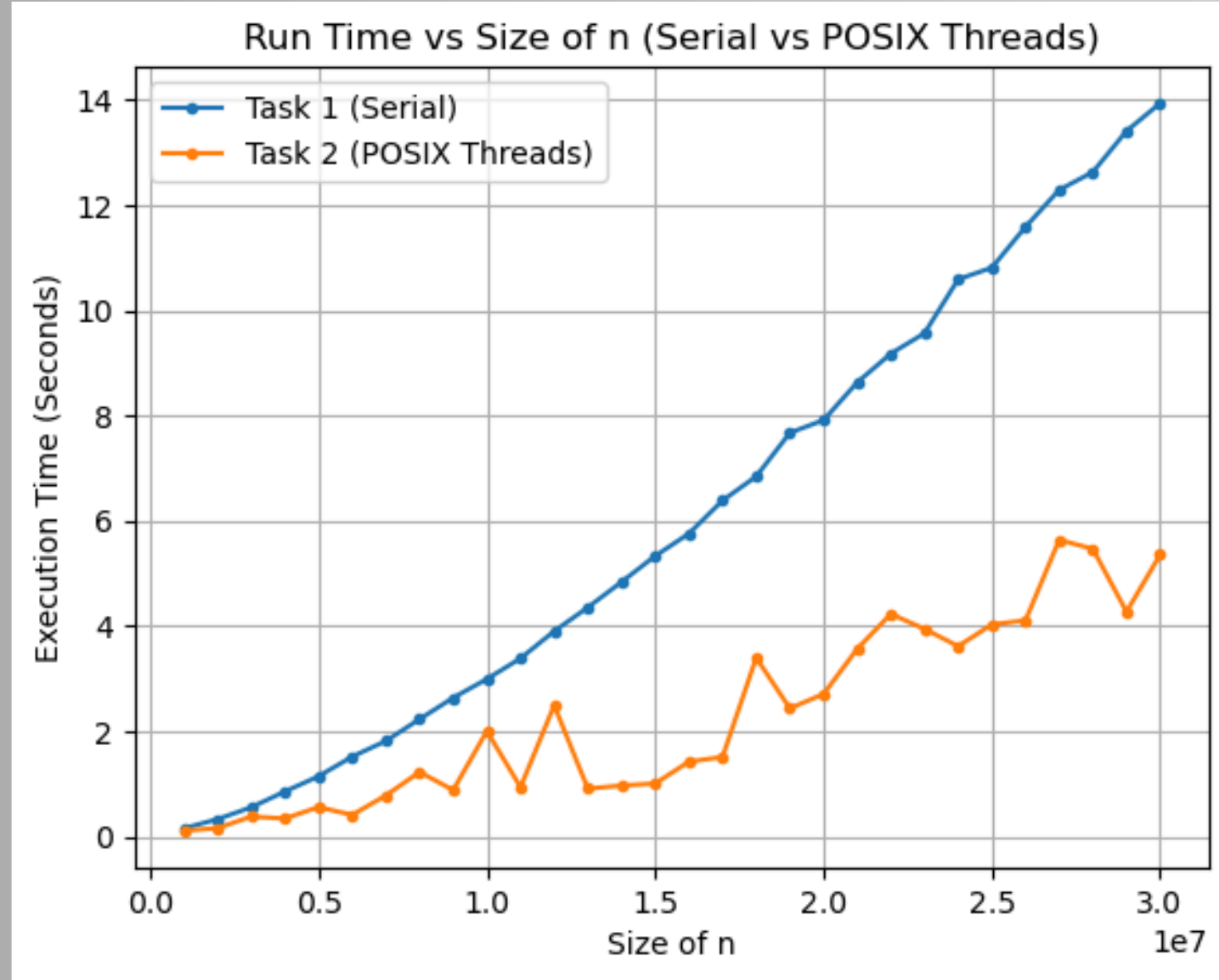
```
{
    pthread_join(threads[t], NULL);
    total_count += local_counts[t];
}
```

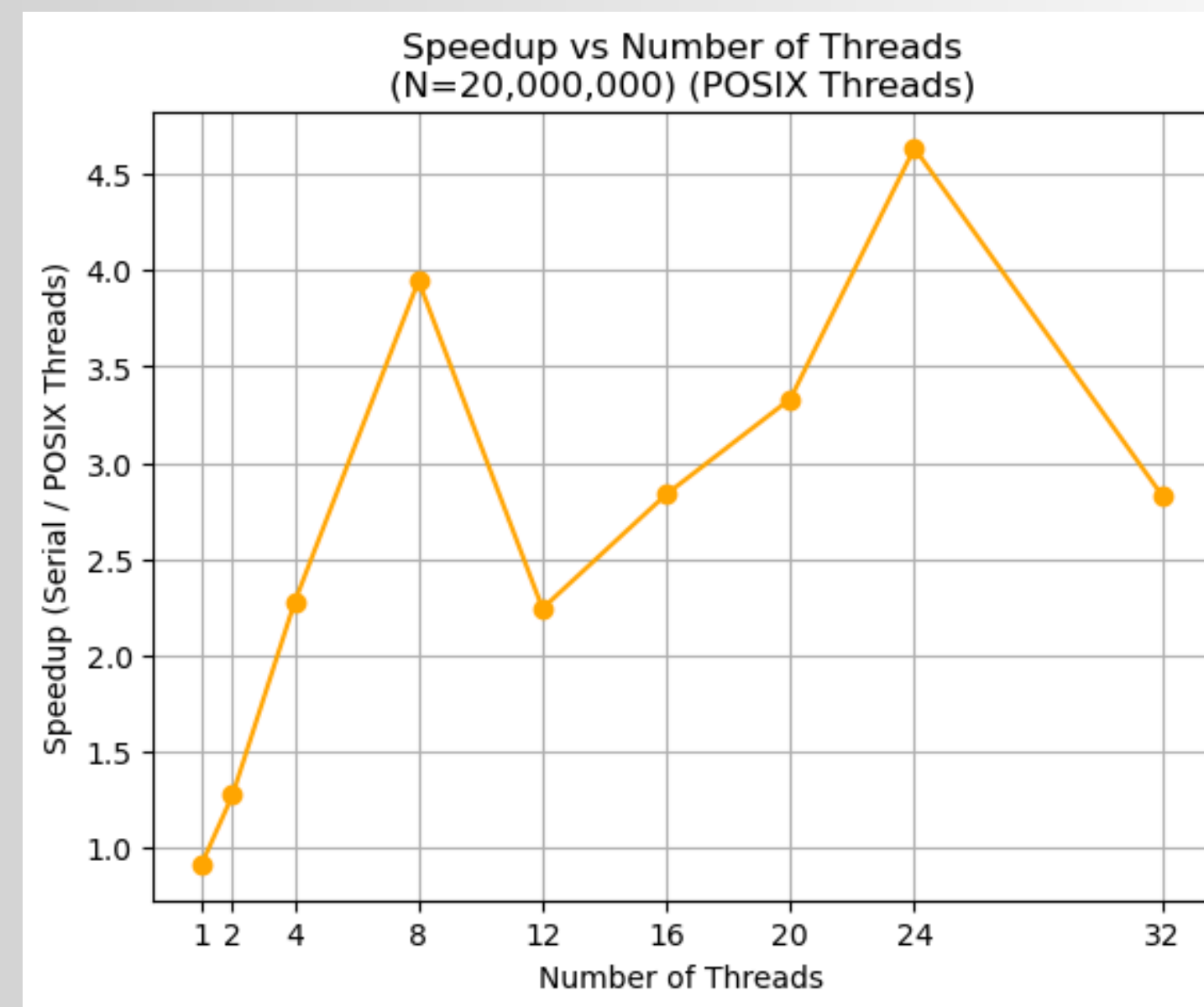
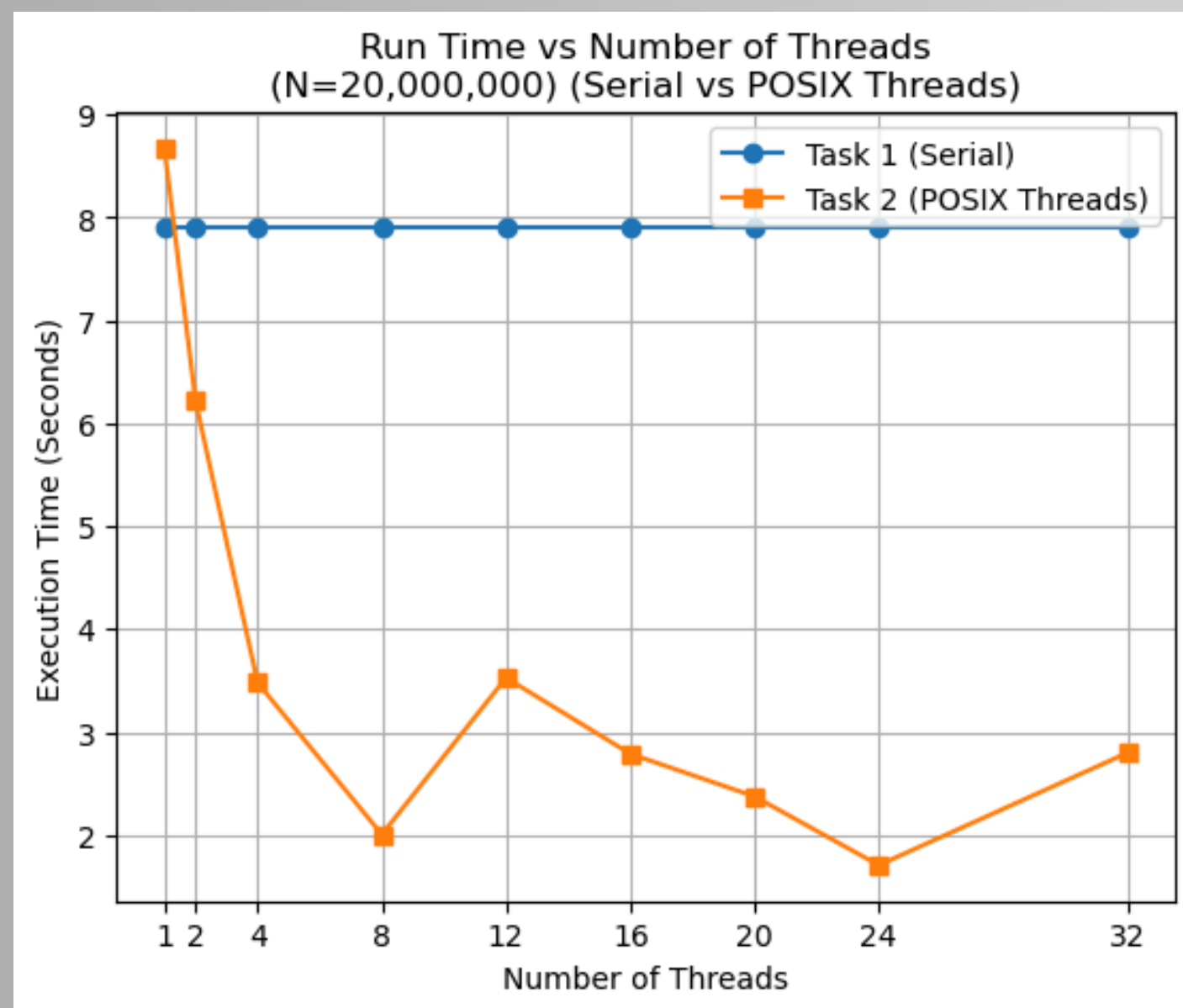
```
// Move each thread's primes together so that they are stored consecutively.
```

```
long long offset = 0;
long long source_offset = 0;
```

```
for (int t = 0; t < num_threads; t++)
{
    if (source_offset != offset)
    {
        memmove(&primes[offset],
                &primes[source_offset],
                local_counts[t] * sizeof(long long));
    }

    offset += local_counts[t];
    source_offset += thread_args[t].end_val - thread_args[t].start_val + 1;
}
```





C

```
long long **local_primes = malloc(max_threads * sizeof(long long *));
long long *local_counts = calloc(max_threads, sizeof(long long));
```

Give each thread local array and count

```
// Allocate a local array for each thread
for (int t = 0; t < max_threads; t++)
{
    local_primes[t] = malloc((limit + 1) * sizeof(long long));
}

// Give each thread numbers to check dynamically.
// Each thread stores its primes in its own local array.
#pragma omp parallel
{
    int thread_id = omp_get_thread_num();

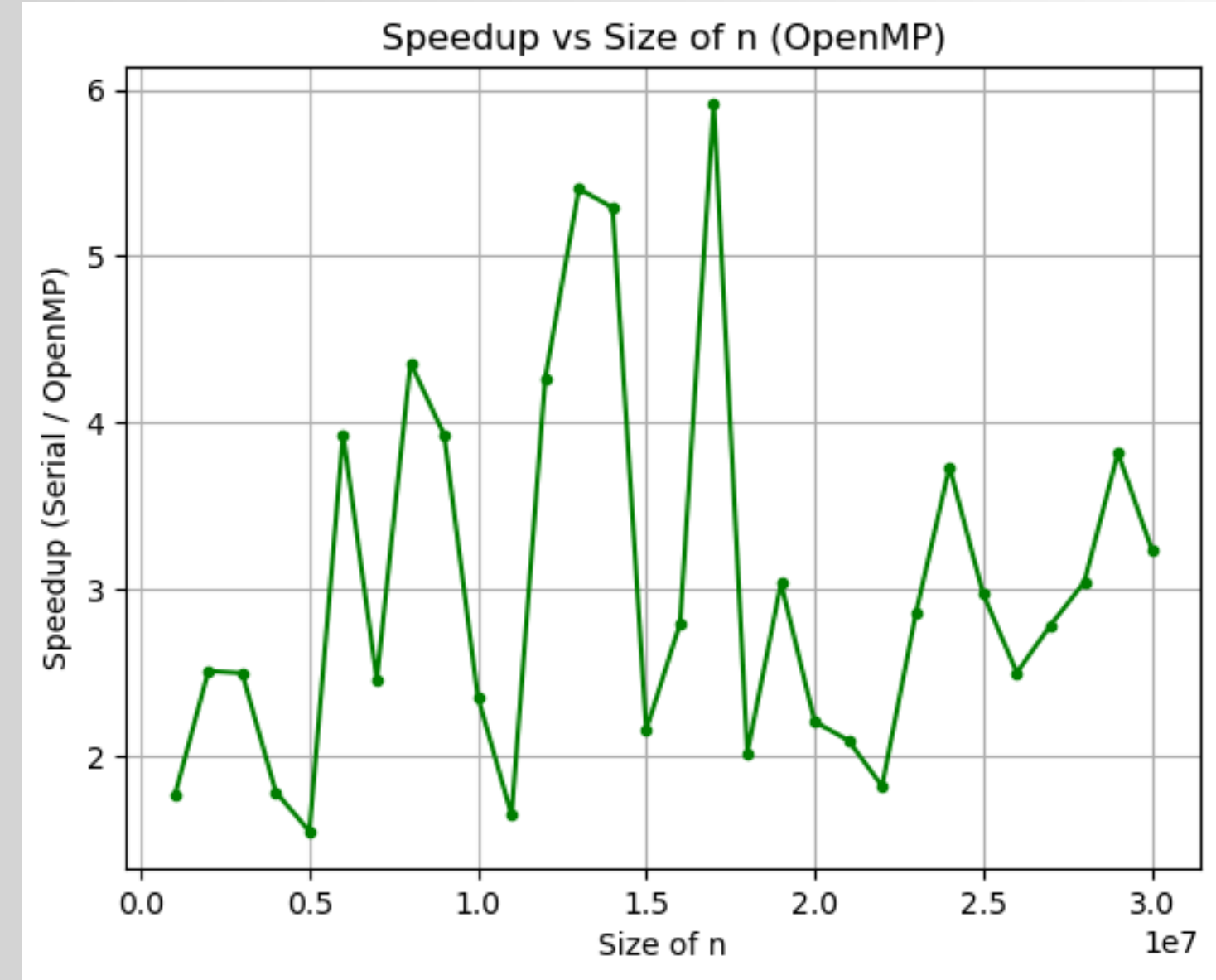
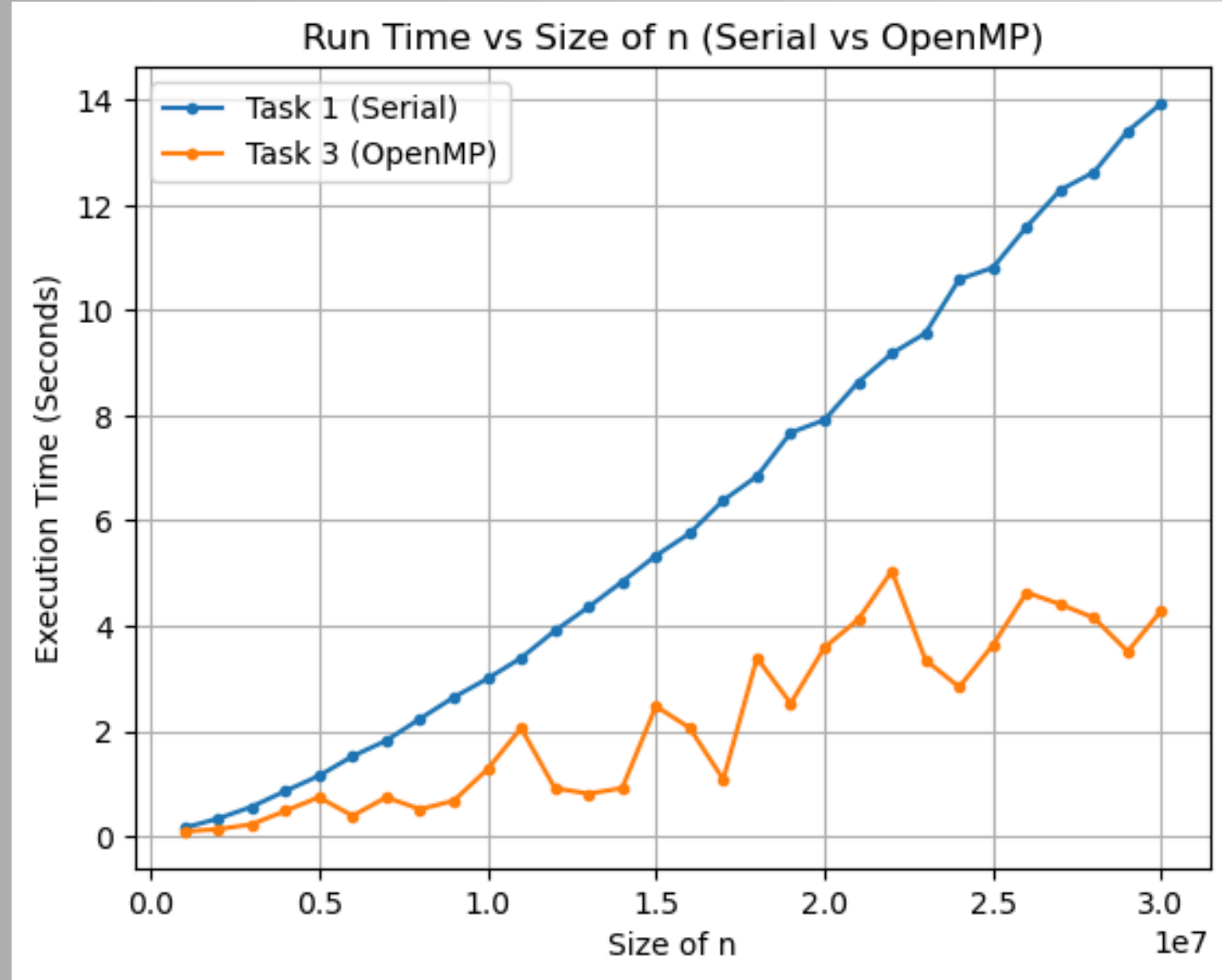
#pragma omp for schedule(dynamic, 1000)
    for (long long i = 2; i <= limit; i++)
    {
        if (is_prime(i))
        {
            local_primes[thread_id][local_counts[thread_id]++] = i;
        }
    }
}
```

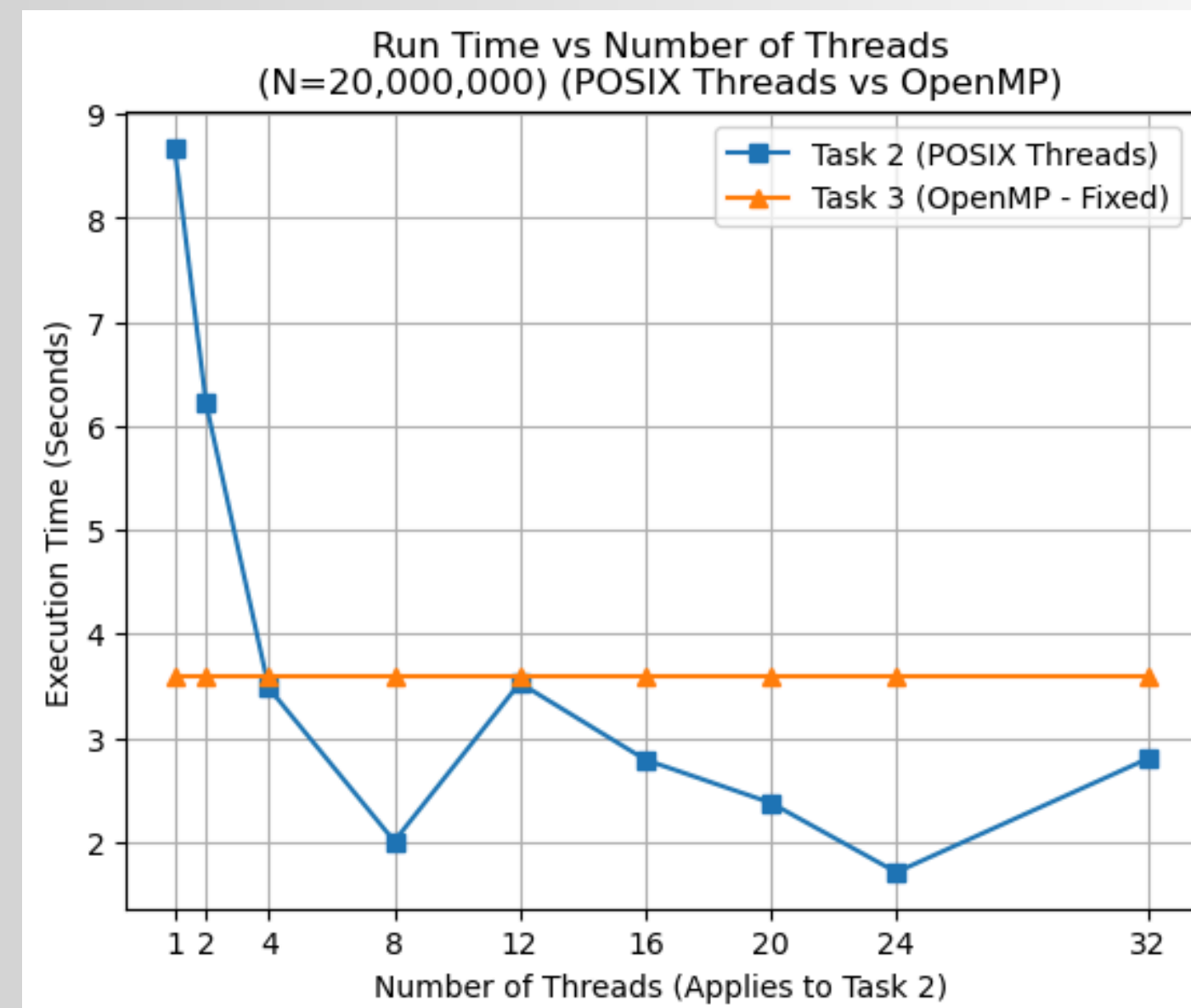
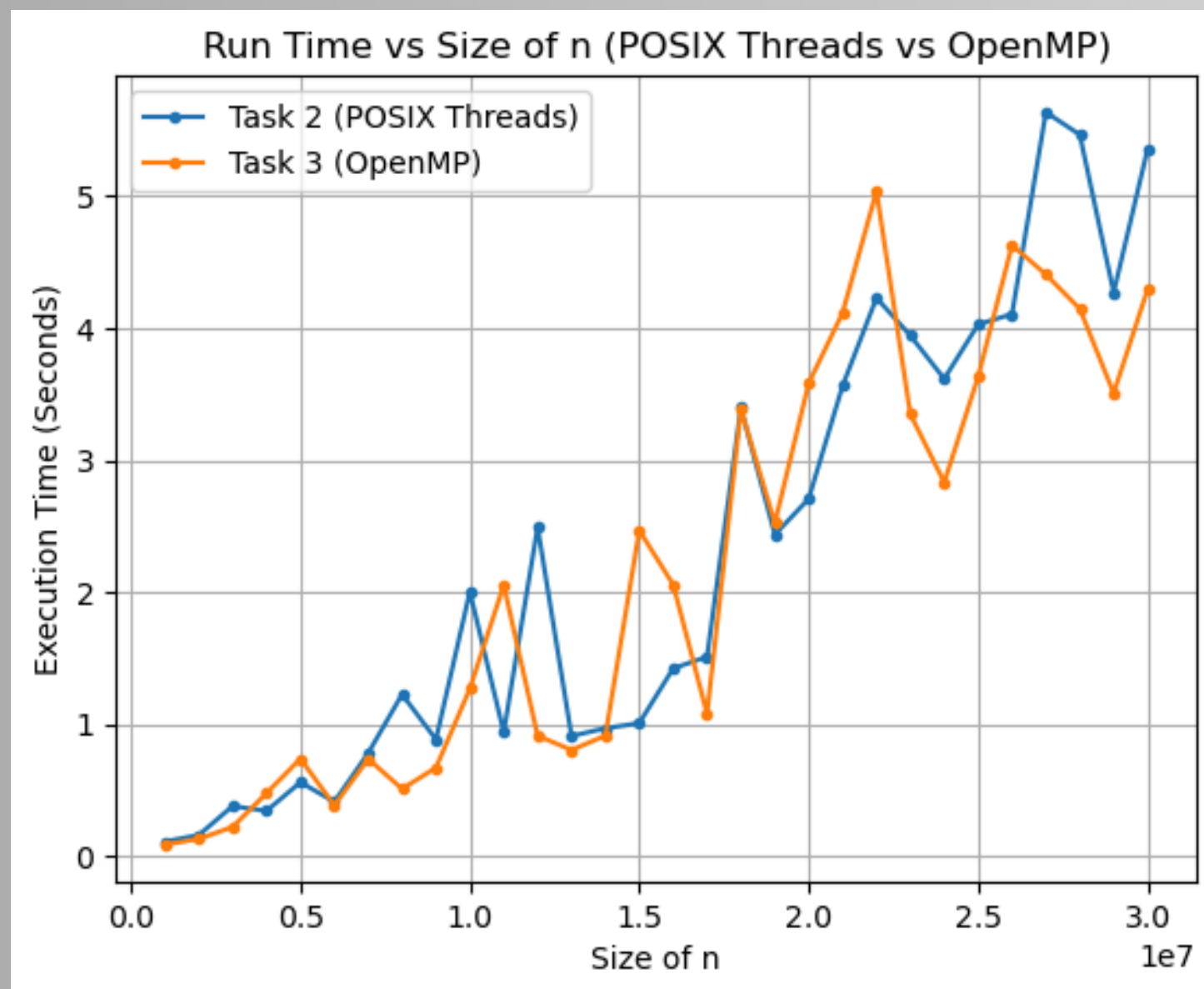
each thread grabs 1000 at a time

add to the local array if it is prime

```
for (int t = 0; t < max_threads; t++)
{
    for (long long i = 0; i < local_counts[t]; i++)
    {
        primes[found_count++] = local_primes[t][i];
    }
}
```

Then combine back to
main array





AI declaration

Statement: AI was used to research, debug, and generate graph

Model: Gemini 3.1 pro

Prompts and usage:

- <https://gemini.google.com/share/d1551da5ffa6?skid=5c57a8ad-660e-48b1-8699-72896e0d9750>
- <https://docs.google.com/document/d/1ILXSUwaHyjR-ZJknT08ccVeWVOuGNCcK-ogk7HC0vp8/edit?usp=sharing>